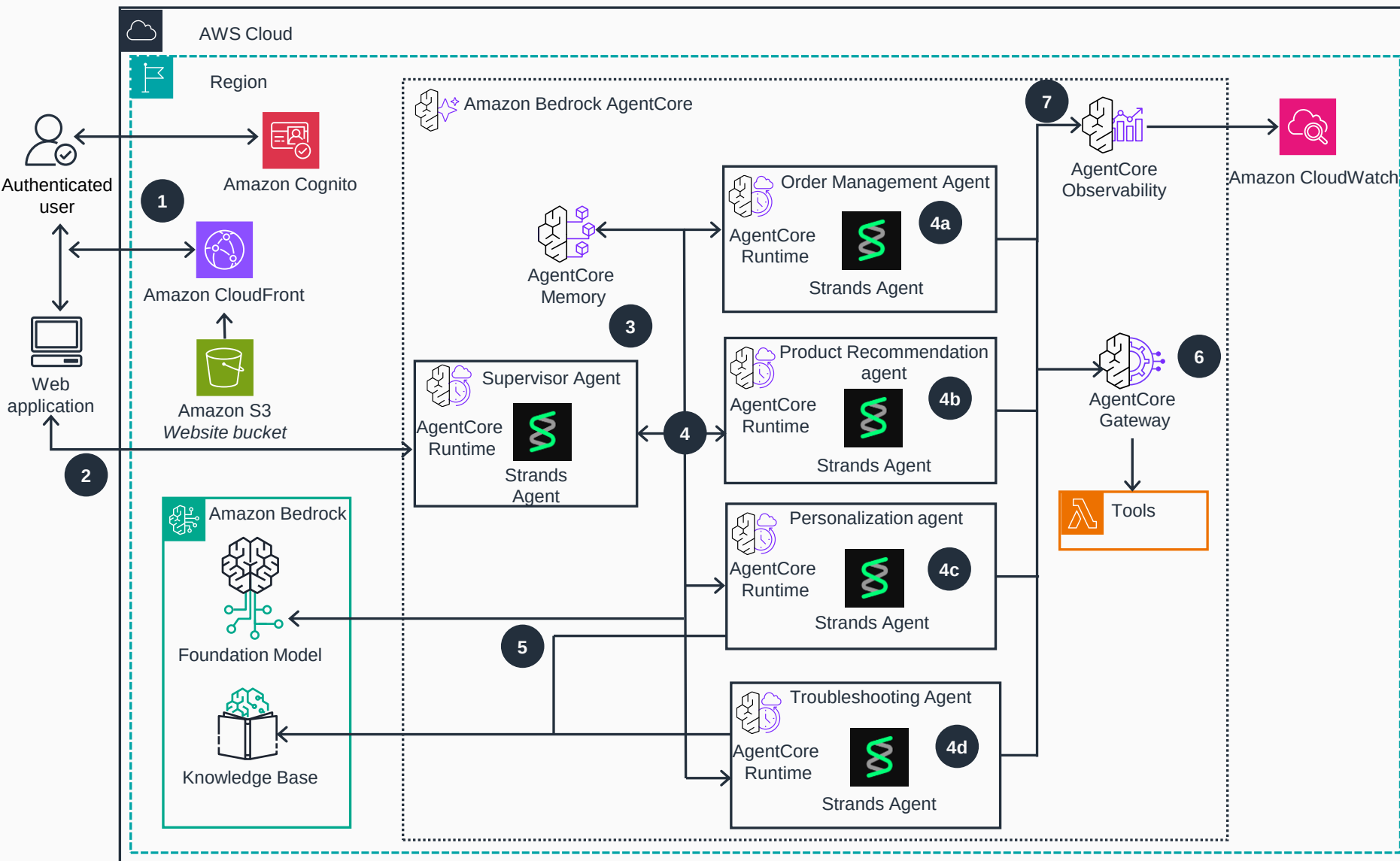


Guidance for Multi-Agent Orchestration on AWS

Amazon Bedrock AgentCore

This architecture diagram showcases Amazon Bedrock AgentCore orchestrating multiple specialized agents, routing customer queries to the right expert for Order Management, Product Recommendations, Personalization, or Troubleshooting. The architecture enables seamless agent collaboration and context sharing for complex customer scenarios. This multi-agent orchestration delivers scalable, personalized customer support by automatically selecting the best specialist agents for each customer need.



1

The user accesses the web application through **Amazon CloudFront**, which delivers static content from the **Amazon Simple Storage Service (Amazon S3)** website bucket. **Amazon Cognito** handles user authentication and authorization, issuing JWT Bearer tokens for API access.

2

After authentication, the web application sends user requests with JWT Bearer tokens directly to the **Amazon Bedrock AgentCore Runtime** endpoint using Server-Sent Events (SSE) for near real-time streaming responses. The **Amazon Bedrock AgentCore Runtime** validates the JWT tokens against **Amazon Cognito's** OIDC discovery endpoint and extracts user identity information including custom customer ID attributes for personalization.

3

The **Amazon Bedrock AgentCore Runtime** forwards the authenticated request to the Supervisor Agent running on Strands Agent, passing along the user context and conversation memory from **Amazon Bedrock AgentCore Memory**.

4

The Supervisor Agent acts as the central orchestrator that analyzes user queries using **Amazon Bedrock** foundation models and determines the appropriate specialized agent to handle the request. Each specialized Strands Sub-Agent receives securely authenticated requests from the Supervisor Agent, ensuring proper user identity verification and maintaining separate conversation contexts for each interaction.

The Supervisor Agent aggregates and coordinates responses from multiple specialized agents when needed, storing all interactions in **Amazon Bedrock AgentCore Memory** for conversation continuity and customer preference learning.

4a

The Order Management Agent running on **Amazon Bedrock AgentCore Runtime** handles order-related queries. It uses **Amazon Bedrock** foundation models for query processing and retrieves order data, inventory information, and shipping details through **Amazon Bedrock AgentCore Gateway** and Tools integration.



Reviewed for technical accuracy May 27, 2025

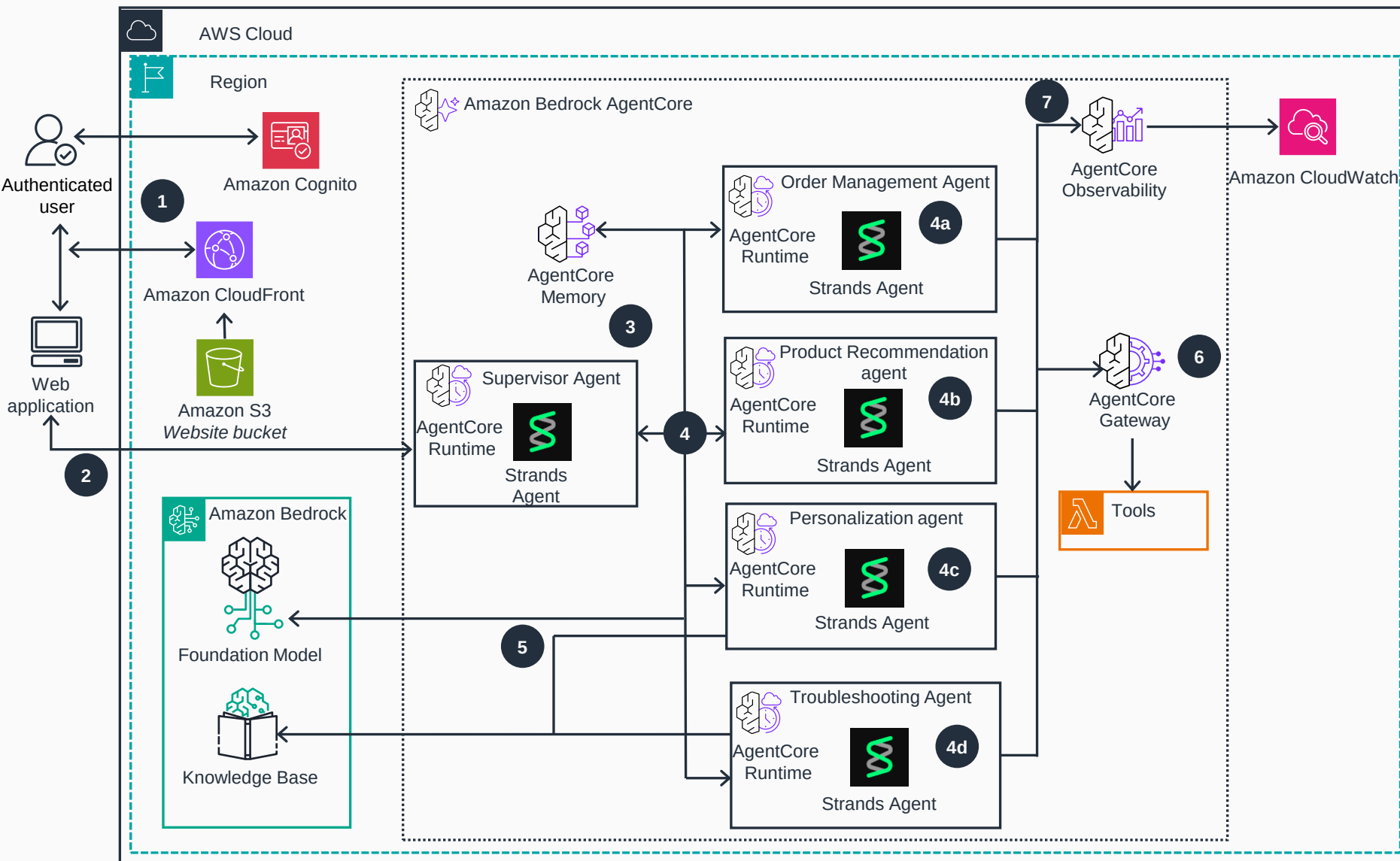
© 2025, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS Reference Architecture

Guidance for Multi-Agent Orchestration on AWS

Amazon Bedrock AgentCore

This architecture diagram showcases Amazon Bedrock AgentCore orchestrating multiple specialized agents, routing customer queries to the right expert for Order Management, Product Recommendations, Personalization, or Troubleshooting. The architecture enables seamless agent collaboration and context sharing for complex customer scenarios. This multi-agent orchestration delivers scalable, personalized customer support by automatically selecting the best specialist agents for each customer need.



4b The Product Recommendation Agent running on **Amazon Bedrock AgentCore** Runtime processes product recommendation requests. It utilizes **Amazon Bedrock** foundation models for natural language understanding and accesses product databases through **Amazon Bedrock AgentCore** Gateway and Tools integration.

4c The Personalization Agent running on **Amazon Bedrock AgentCore** Runtime provides personalized responses across multiple customer sessions. It leverages **Amazon Bedrock** foundation models for personalization logic and connects to **Amazon Bedrock Knowledge Base** to analyze customer browsing patterns. It uses Retrieval Augmented Generation (RAG) to customize model responses with user-specific data and behavioral insights.

4d The Troubleshooting Agent running on **Amazon Bedrock AgentCore** Runtime handles technical support queries. It uses **Amazon Bedrock** foundation models for understanding technical issues and connects to **Amazon Bedrock Knowledge Base** containing FAQs, troubleshooting guides, and warranty information through RAG capabilities.

5 The specialized agents invoke **Amazon Bedrock** foundation models for natural language understanding, query processing, and response generation specific to their domain expertise. For knowledge-intensive queries, the specialized agents access **Amazon Bedrock Knowledge Bases** using semantic search with embedding models to retrieve relevant information including customer browsing history, product catalogs, FAQs, and troubleshooting guides.

6 The specialized agents utilize **Amazon Bedrock AgentCore** Gateway and **AWS Lambda** function Tools integration to access external systems, databases, and APIs for near real-time data retrieval such as order status, inventory levels, and product information.



Reviewed for technical accuracy May 27, 2025

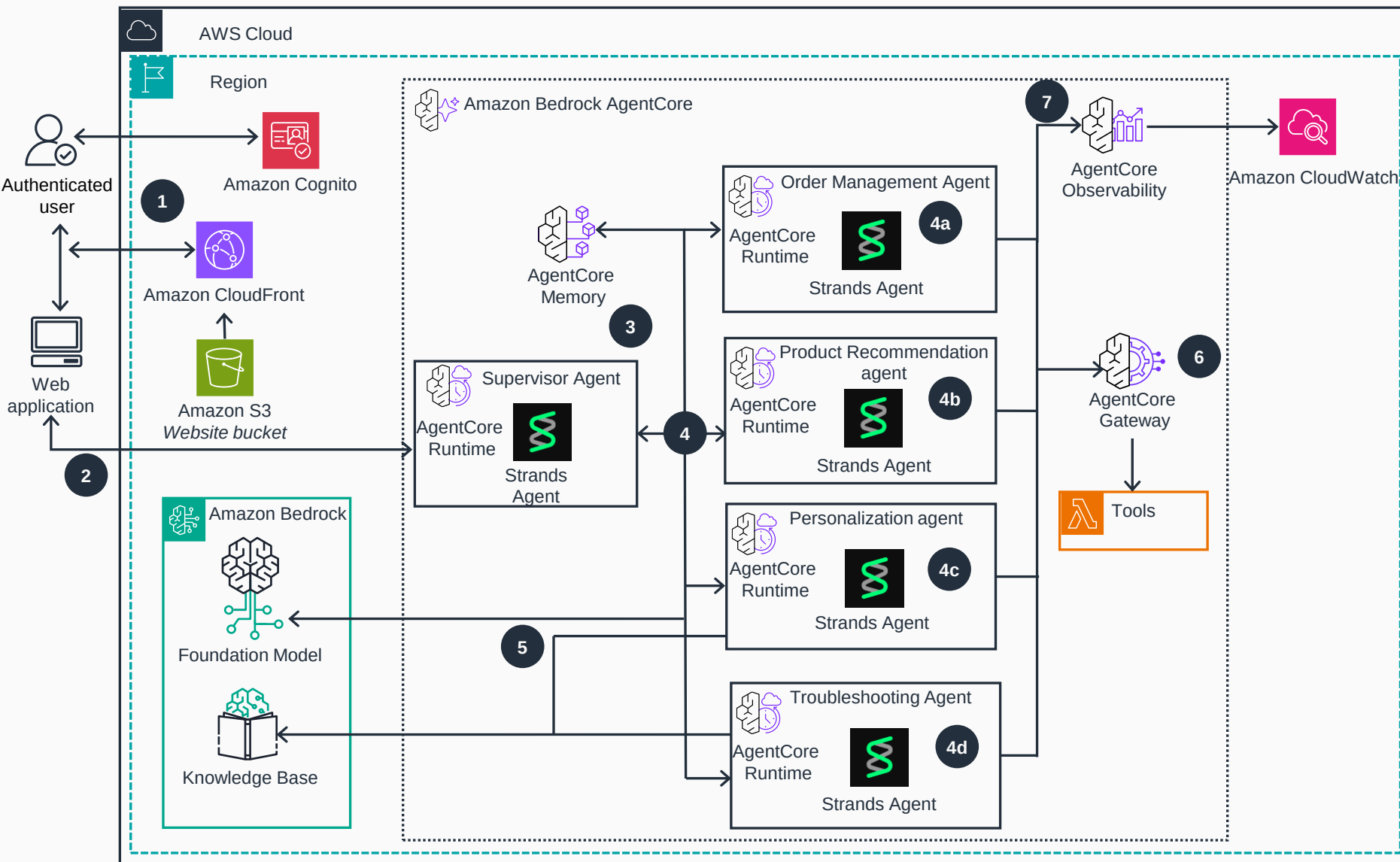
© 2025, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS Reference Architecture

Guidance for Multi-Agent Orchestration on AWS

Amazon Bedrock AgentCore

This architecture diagram showcases Amazon Bedrock AgentCore orchestrating multiple specialized agents, routing customer queries to the right expert for Order Management, Product Recommendations, Personalization, or Troubleshooting. The architecture enables seamless agent collaboration and context sharing for complex customer scenarios. This multi-agent orchestration delivers scalable, personalized customer support by automatically selecting the best specialist agents for each customer need.



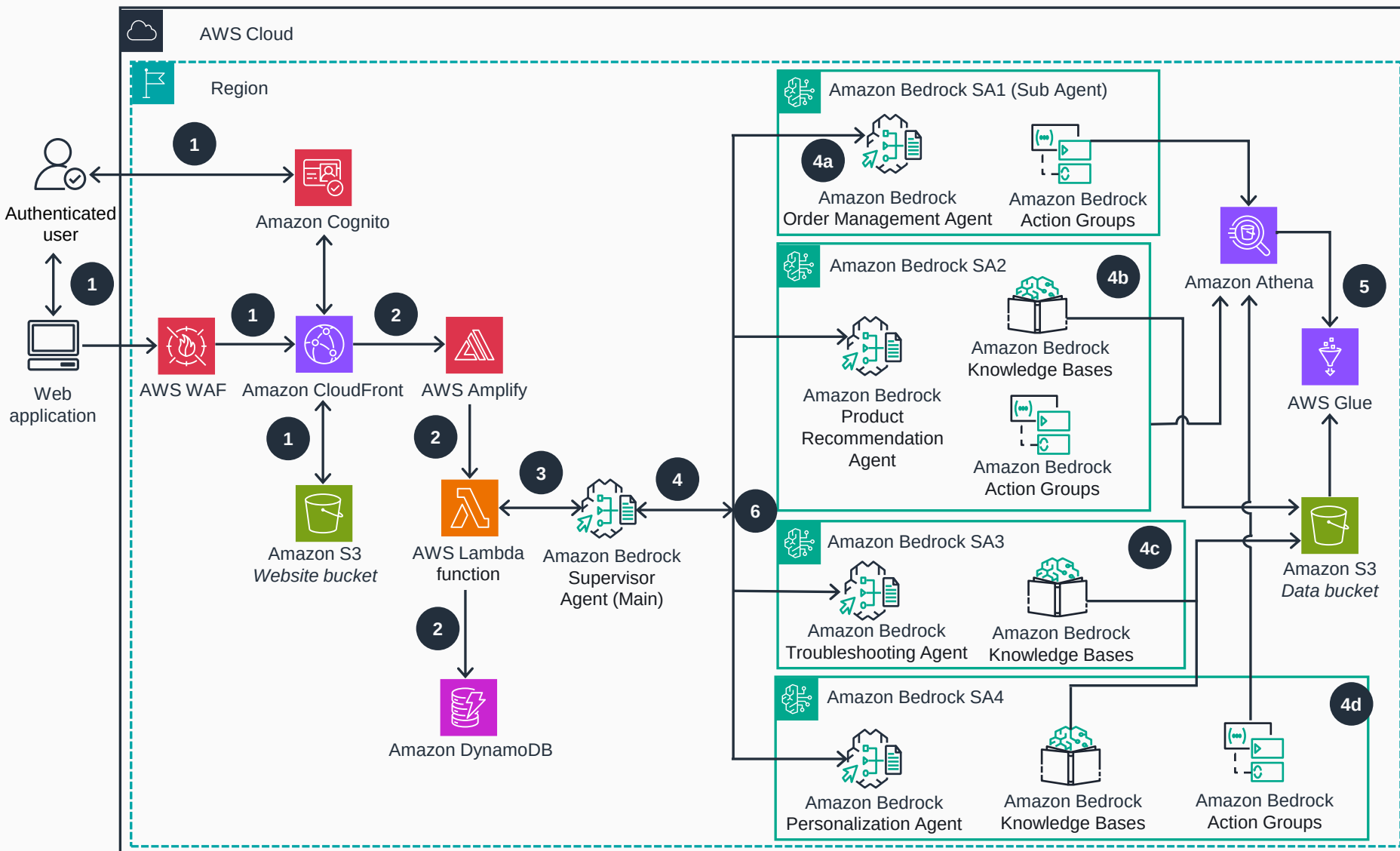
7 **Amazon Bedrock AgentCore** Observability captures comprehensive monitoring data including agent performance metrics, conversation flows, tool usage patterns, and error rates, while **Amazon CloudWatch** logs detailed execution traces for continuous system optimization and debugging.



Guidance for Multi-Agent Orchestration on AWS

Using Amazon Bedrock Multi-Agent Collaboration Feature

This architecture diagram shows how to build an intelligent customer support system using Amazon Bedrock multi-agent collaboration, where specialized agents work together to deliver data-driven customer support.



1 The user accesses the web application through [AWS WAF](#) and [Amazon CloudFront](#) which delivers content from the [Amazon Simple Storage Service \(Amazon S3\)](#) website bucket. [Amazon Cognito](#) handles authentication.

2 After authentication, user requests are sent to [AWS Amplify](#), which serves as the entry point for all interactions. **Amplify** validates the request and routes it to the appropriate [AWS Lambda](#) function for processing, maintaining a secure and scalable communication channel. [Amazon DynamoDB](#) stores session data.

3 The **Lambda** function processes the incoming request and communicates with the [Amazon Bedrock](#) Supervisor Agent (Main).

4 The **Amazon Bedrock** Supervisor Agent (Main) analyzes the user query to determine intent and routes it to the appropriate sub agent. This central orchestrator maintains context across the conversation and ensures requests are handled by the most suitable sub agent.

4a For order-related queries, the Order Management Agent retrieves data from the order management database in [Amazon Athena](#), accessing orders and inventory tables through its action groups. The action groups execute SQL queries and format structured responses about order status, shipping details, and inventory availability.

4b When product recommendations are needed, this specialized agent accesses the product recommendation database in **Athena** while its knowledge base provides unstructured customer feedback data from **Amazon S3**. Action groups perform recommendation algorithms and format product suggestions with relevant details.

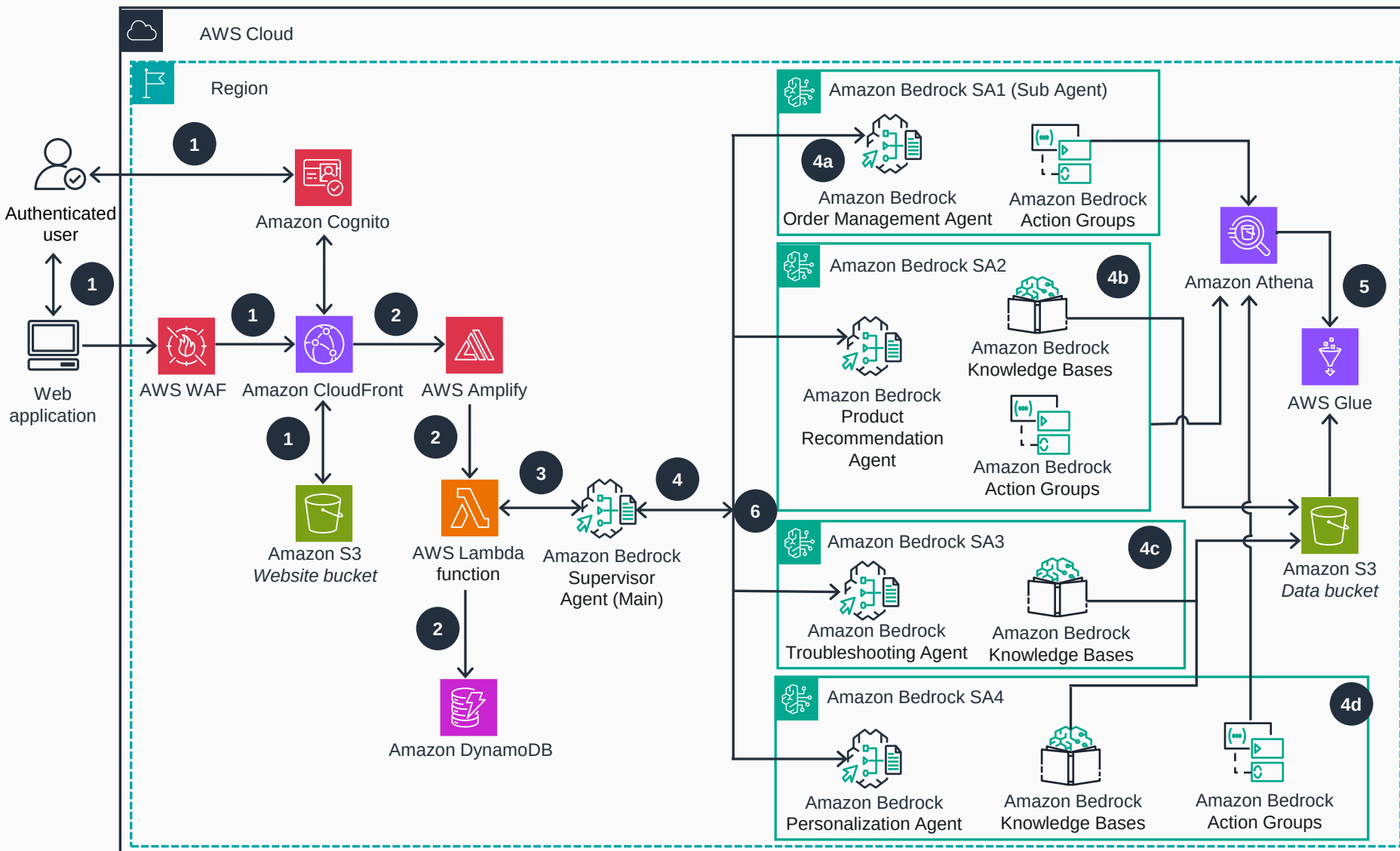
4c For technical issues, the Troubleshooting Agent accesses its knowledge base containing FAQs and Troubleshooting Guide document collections. It uses vector search capabilities to match customer issues with relevant troubleshooting content and to retrieve step-by-step solutions without requiring action groups.



Guidance for Multi-Agent Orchestration on AWS

Using Amazon Bedrock Multi-Agent Collaboration Feature

This architecture diagram shows how to build an intelligent customer support system using Amazon Bedrock multi-agent collaboration, where specialized agents work together to deliver data-driven customer support.



4d For personalization needs, the Personalization Agent accesses the personalization database in **Athena**, querying the customer's preferences table through action groups. These action groups execute tailored SQL queries, perform preference analysis, and format responses. Its knowledge base contains browser history data from **Amazon S3** that reveals actual customer behavior patterns, complementing the structured data to create a comprehensive view of individual customer profiles and past interactions.

5 Sub agents construct and execute SQL queries against **Athena**, which uses the [AWS Glue Data Catalog](#) to understand the schema and location of data, then queries the data directly in **Amazon S3** without requiring data movement or transformation.

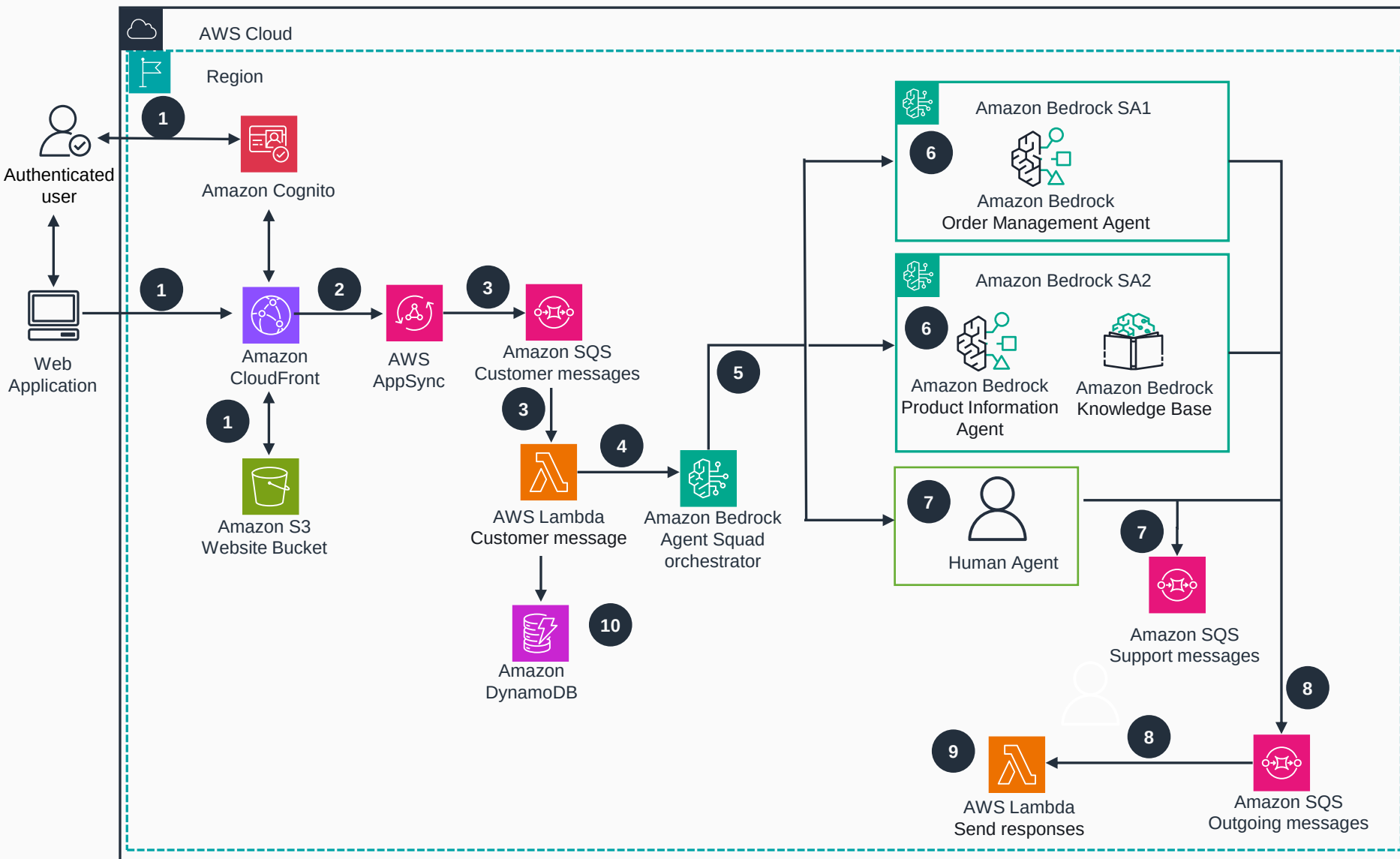
6 After gathering necessary information from databases and knowledge bases, the sub agents generate a comprehensive response. This response is then sent back through the Supervisor Agent to the **Lambda** function, then **Amplify**, and finally to the user's web interface.



Guidance for Multi-Agent Orchestration on AWS

Using Agent Squad

This architecture diagram shows how to build an intelligent customer support system using Amazon Bedrock multi-agent collaboration, where specialized agents work together to deliver data-driven customer support.



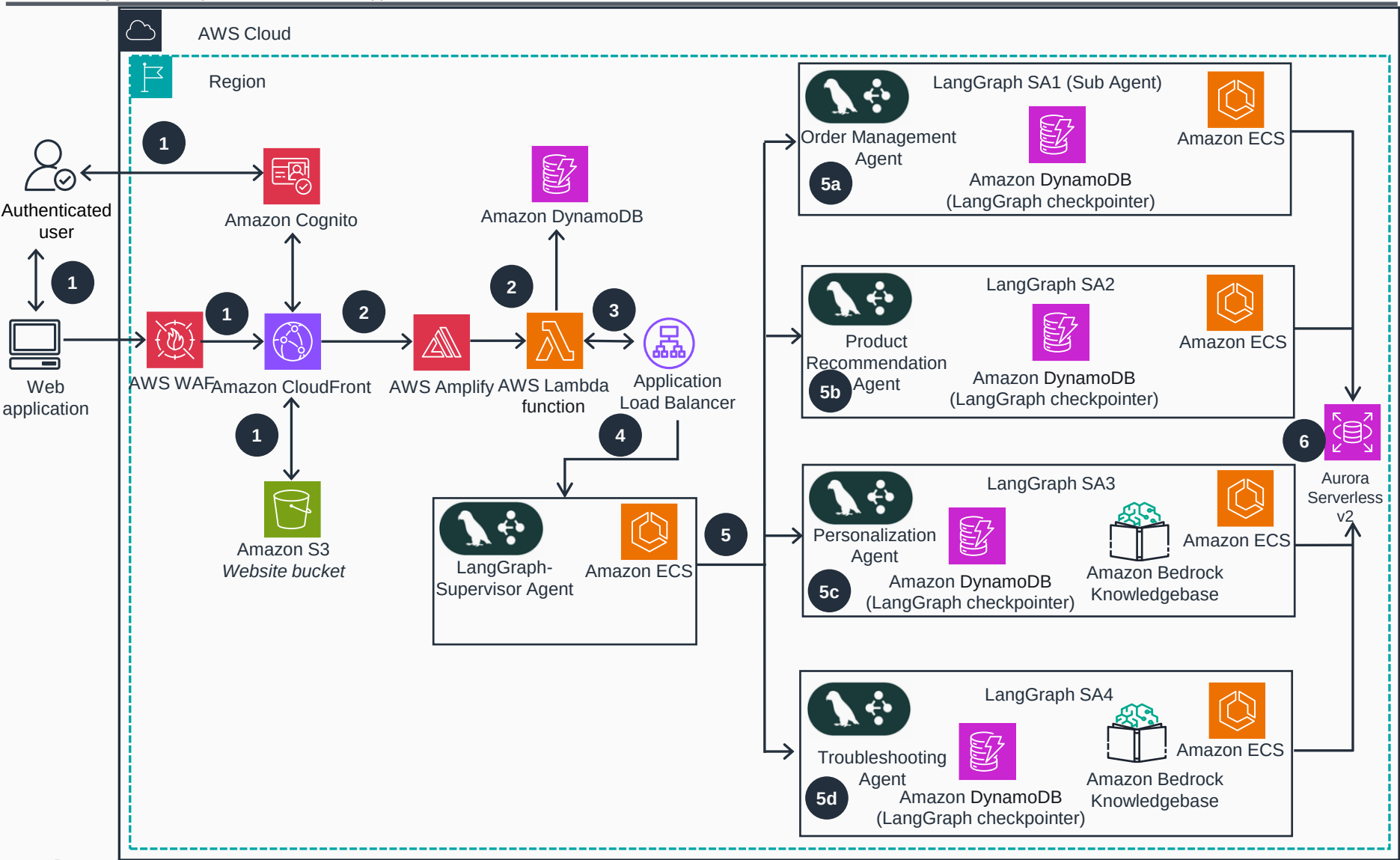
- 1 The user accesses the web application through **Amazon CloudFront**, which delivers content from the **Amazon S3** Website bucket, while **Amazon Cognito** handles authentication and provides temporary credentials via Identity Pool.
- 2 The authenticated user sends messages through the web UI, which communicates with **AWS AppSync** GraphQL API to process queries, mutations, and subscriptions for real-time communication.
- 3 **AWS AppSync** routes customer messages to the **Amazon SQS** Customer Messages Queue, which triggers the **AWS Lambda** Customer Message Handler function to process incoming customer inquiries.
- 4 **AWS Lambda** Customer Messages initializes the **Agent Squad orchestrator**, which uses **Amazon Bedrock** Classifier to analyze the message content and determine which specialized AI agent should handle the request.
- 5 Based on the classification, the message is routed to one of three agents: the Order Management Agent (using Claude 3 Sonnet), the Product Information Agent (using Claude 3 Haiku), or the Human Agent for complex cases requiring human intervention.
- 6 The Order Management Agent handles order-related inquiries using tools for order lookup, shipment tracking, and return processing, while the Product Information Agent retrieves product details from the Amazon Bedrock Knowledge Base connected to OpenSearch Serverless.
- 7 When complex inquiries are routed to the Human Agent, it sends the customer message to the Amazon SQS Support Messages Queue and notifies the customer that their request will be handled by a human support representative.
- 8 Agent responses are sent to the Amazon SQS Outgoing Messages Queue, which triggers the AWS Lambda Send Response Handler to process and format the responses.
- 9 The Send Response Lambda sends the formatted responses back to AWS AppSync, which delivers them to the web UI through GraphQL subscriptions for real-time updates to the customer interface.
- 10 Throughout this process, Amazon DynamoDB stores conversation history and session data, while AWS IAM roles and policies ensure secure access to all services and resources.



Guidance for Multi-Agent Orchestration on AWS

LangGraph

This architecture showcases a LangGraph-powered supervisor agent running on Amazon ECS that intelligently coordinates four specialized sub-agents (Order Management, Product Recommendation, Personalization, and Troubleshooting) through LangGraph's agent orchestration framework, enabling seamless task delegation, context sharing, and response synthesis across distributed agents for comprehensive customer support scenarios.



1

The user accesses the web application through **AWS WAF** and **Amazon CloudFront** which delivers content from the **Amazon Simple Storage Service (Amazon S3)** website bucket. **Amazon Cognito** handles user authentication and authorization.

2

After authentication, user requests are sent to **AWS Amplify**, which serves as the entry point for all interactions. Amplify validates the request and routes it to the **AWS Lambda** resolver function for processing. **Amazon DynamoDB** stores session data and user interaction history.

3

The **AWS Lambda** function processes the incoming request and forwards it to the **Application Load Balancer**, which distributes the traffic for high availability and optimal performance.

4

The **Application Load Balancer** routes the request to the **LangGraph Supervisor Agent** running on **Amazon ECS**. This supervisor agent acts as the central orchestrator that analyzes user queries and determines the appropriate specialized agent to handle the request.

5

The **LangGraph Supervisor Agent** analyzes the user query to determine intent and routes it to the appropriate specialized sub-agent based on the request type:

5a

LangGraph SA1 (Order Management Agent) running on **Amazon ECS** handles order-related queries. It uses **Amazon DynamoDB** based checkpoint saver implementation for LangGraph to maintain conversation context and retrieves order data, inventory information, and shipping details.

5b

LangGraph SA2 (Product Recommendation Agent) running on **Amazon ECS** processes product recommendation requests. It accesses product databases through **Amazon DynamoDB** based checkpoint saver implementation for LangGraph and uses machine learning algorithms to provide personalized product suggestions.



Reviewed for technical accuracy May 27, 2025

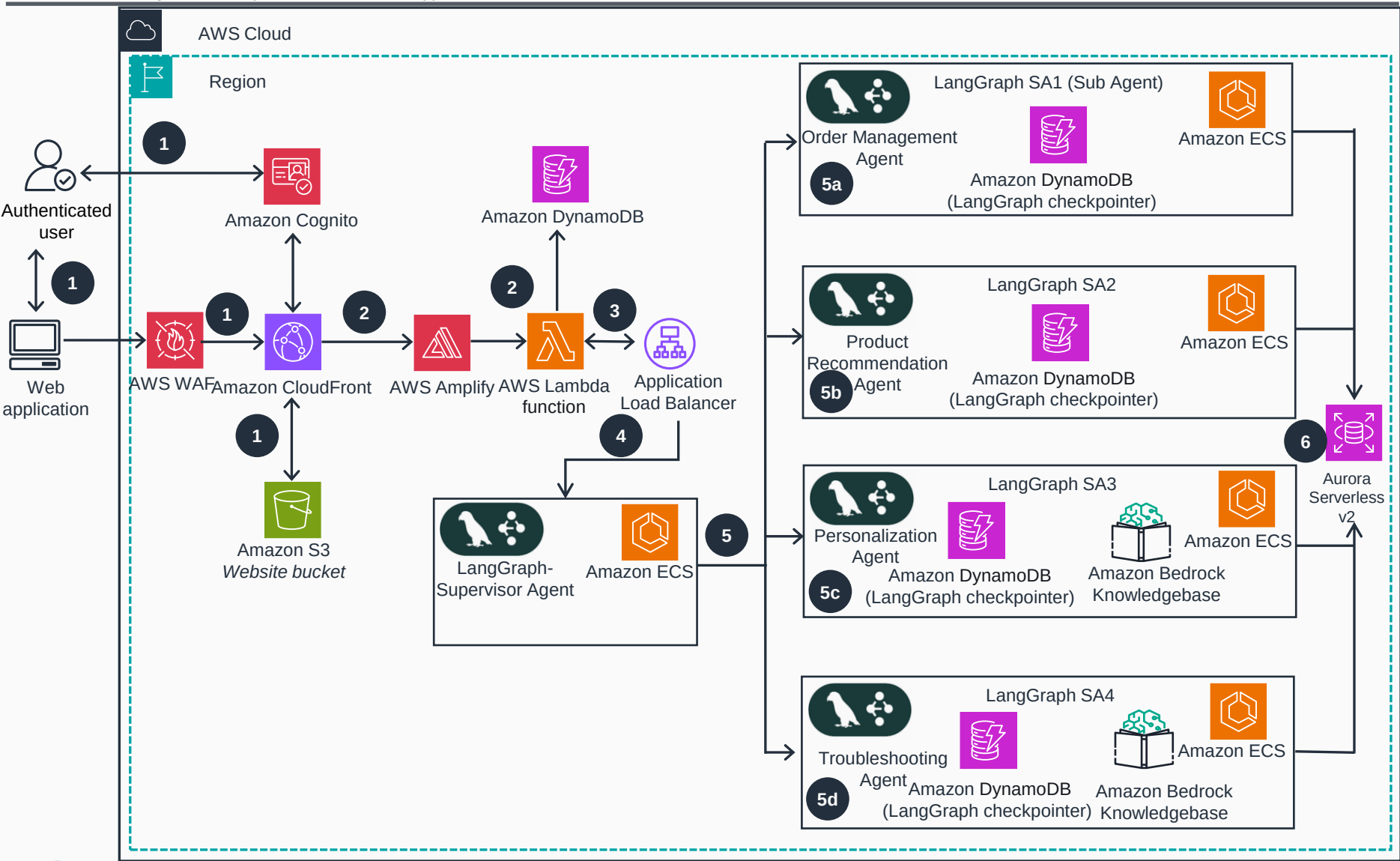
© 2025, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS Reference Architecture

Guidance for Multi-Agent Orchestration on AWS

LangGraph

This architecture showcases a LangGraph-powered supervisor agent running on Amazon ECS that intelligently coordinates four specialized sub-agents (Order Management, Product Recommendation, Personalization, and Troubleshooting) through LangGraph's multi-agent collaboration framework, enabling seamless task delegation, context sharing, and response synthesis across distributed agents for comprehensive customer support scenarios.



5c

LangGraph SA3 (**Personalization Agent**) running on **Amazon ECS** manages user-specific customization and personalization requests. It connects to **Amazon Bedrock Knowledgebase** to access customer preference data and behavioral patterns.

5d

LangGraph SA4 (**Troubleshooting Agent**) running on **Amazon ECS** handles technical support queries. It accesses the **Amazon Bedrock Knowledgebase** containing FAQs, troubleshooting guides, and technical documentation to provide step-by-step solutions.

6

Aurora Serverless v2 provides persistent relational database storage for structured data including user profiles, order history, product catalogs, and system metadata that the agents can query as needed.



Reviewed for technical accuracy May 27, 2025

© 2025, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS Reference Architecture